



[GIS] 🌸 포트폴리오 주제_기능_기술 스택 정리_3차

👤 생성자	👤 김탱
🕒 생성 일시	2026년 8월 4일 오전 8:53
☰ 카테고리	프로젝트
🕒 최종 업데이트 시간	2026년 8월 6일 오후 1:26
☰ 도구	
✨ 작성상태	작성완료

GIS 주제 : 아이·노인 케어 위치추적 알림 시스템

🌸 프로젝트 개요

목표

보호 대상자의 위치를 실시간으로 추적하고, 보호자에게 위치 정보 및 위험 상황 알림을 제공하는 케어 시스템 구축.

기획 의도와 서비스 방향

아이와 노인의 위치·이동 데이터를 기반으로 보호 대상자의 안전을 자동 모니터링하고, 보호자의 상시 확인 부담을 줄이는 AI 기반 GIS 케어 서비스

핵심기능

- 사용자 인증 - 보호자 / 보호대상자 관리
- 실시간 위치 추적 -GeoFence 기반 위치 이벤트 감지
- WebSocket 실시간 통신
- AI 기반 예측기능

- LLM 기반 AI Care Assistant
- 알림 시스템

주요 기능 요약

위치 확인 → 도착/이탈 판단 → 이상 이동 감지 → 알림 → AI 분석/도우미

기술스택

▼ 표

구분	기술	사용 목적
Frontend	Flutter	Android / iOS 크로스 플랫폼 모바일 앱 개발
	Dart	Flutter 개발 언어
	Google Maps SDK	지도 표시 및 실시간 위치 시각화
	WebSocket	실시간 위치 정보 수신 및 지도 갱신
Backend	Java	서버 개발 언어
	Spring boot	REST API 및 WebSocket 기반 메인 백엔드 서버 구현
	Spring Security	인증(Authentication) 및 인가(Authorization), 접근 제어
	Spring Data JPA	객체-관계 매핑(ORM) 및 데이터 접근 계층 구현
	Hibernate	JPA 구현체 및 영속성 관리
	JWT	Access Token 기반 사용자 인증 및 API 접근 제어
	OAuth 2.0 (Google)	Google 간편 로그인 및 소셜 인증
AI Server	Python	AI / Machine Learning 서버 개발 언어
	FastAPI	AI Prediction API 및 LLM 연동 API 서버 구현
	Uvicorn	FastAPI 실행을 위한 ASGI 서버
	Pydantic	Request/Response DTO 및 데이터 검증
Database	PostgreSQL	사용자, 장소, 방문이력, 위치이력 등 영구 데이터 저장

구분	기술	사용 목적
	pgvector	AI Vector Embedding 저장 및 유사도 검색
	Redis	Cache Aside 전략, Refresh Token, JWT Blacklist 및 캐시 관리
AI / Machine Learning	Pandas	GPS 데이터 전처리 및 Feature 생성
	NumPy	Feature 수치 계산 및 데이터 처리
	Scikit-learn	데이터 전처리, Label Encoding, Train/Test Split 및 모델 성능 평가
	XGBoost	방문 장소 예측 모델 학습
	LightGBM	방문 장소 예측 모델 학습 및 성능 비교
	Hugging Face Spaces	학습된 <code>model.pkl</code> (ML 모델)서빙 및 Prediction API 제공
	LLM API (OpenAI / Gemini)	AI 케어 비서, 자연어 질의응답 및 이동 리포트 생성
Notification	Firebase Cloud Messaging (FCM)	도착 알림, 재알림, 긴급 알림, AI 이상행동 알림 Push 전송
External API	Google Places API	GPS 좌표를 장소명으로 변환 및 장소 정보 조회
	Google Maps API	지도 및 위치 정보 제공
Infrastructure	AWS EC2 (Ubuntu)	서비스 운영 서버
	Docker	Spring Boot / FastAPI 애플리케이션 컨테이너 실행
	Docker Compose	Backend, AI Server, PostgreSQL, Redis 등 서비스 통합 관리
	Nginx	Reverse Proxy 및 HTTPS 처리
CI/CD	Jenkins	자동 빌드 및 배포 파이프라인 구축
	GitHub Webhook	Git Push 이벤트 감지 및 Jenkins 연동
Version Control	Git	형상 관리
	GitHub	원격 저장소 및 협업 관리

주요 기술 적용 포인트

- **OAuth2 + JWT** 기반 인증 및 Role(Guardian / CareTarget) 기반 권한 관리

- **WebSocket**을 활용한 보호대상자 실시간 위치 추적
- **GeoFence** 기반 도착/출발 자동 감지 및 방문 히스토리 생성
- **Redis Cache Aside Pattern**을 적용하여 장소 정보, 사용자 정보, AI 예측 결과 캐싱
- **XGBoost / LightGBM** 기반 방문 장소 예측 AI 구현
- **LLM API**를 활용한 AI 케어 비서 및 자연어 기반 이동 리포트 제공
- **Firestore Cloud Messaging(FCM)** 기반 실시간 Push 알림 서비스
- **Docker Compose + Jenkins + GitHub Webhook** 기반 CI/CD 자동 배포 환경 구축

▼ 상세

- **GeoFence**

GeoFence는 GPS 좌표를 기반으로 특정 위치에 가상의 반경(가상 울타리)을 생성하고, 사용자가 해당 영역에 **진입(Enter)** 또는 **이탈(Exit)** 하는 이벤트를 감지하는 위치 기반 기술

- **사용목적** : 보호자가 등록한 장소(집, 학교, 병원, 복지관 등)에 도착, 출발을 자동으로 판단하기 위해
- **적용기능**
 - 도착알림
 - 도착 확인 요청
 - 이상 이동 패턴 감지
 - 방문 히스토리 생성 : 도착시간 & 출발시간 저장
 - AI 방문 예측 학습 데이터 생성

- **WebSocket**

WebSocket은 클라이언트와 서버가 연결을 유지한 상태에서 양방향으로 실시간 데이터를 송수신할 수 있는 통신 프로토콜

HTTP 방식은 요청-응답 구조는 실시간 통신에 비효율적, 웹소켓을 사용하여 실시간 양방향 통신

- **사용목적** : 보호자의 지도에서 보호대상자의 위치를 실시간으로 갱신하기 위해 사용

- **적용기능**

- 실시간 위치 추적
- 지도 마커 실시간 갱신
- 위치 변경 이벤트 전송
- 이상행동 실시간 감지

- **Redis Cache**

edis는 메모리(In-Memory) 기반의 Key-Value 저장소로, 디스크 기반 데이터베이스보다 매우 빠른 조회 성능을 제공하는 캐시 시스템

- **사용목적**

자주 조회되는 데이터를 메모리에 저장하여 데이터베이스 부하를 줄이고 응답 속도를 향상 효과 기대

- **적용기능**

- 장소 목록 캐시
- 사용자 정보 캐시
- AI 방문 예측 결과 캐시
- Google Places API 결과 캐시
- FCM Token 저장
- Refresh Token 저장
- JWT Blacklist 관리

- **OAuth**

OAuth는 외부 인증 서비스를 이용하여 사용자를 인증하는 표준 인증 프로토콜로 사용자는 Google 계정으로 로그인하며, 서비스는 사용자의 비밀번호를 직접 저장하거나 관리하지 않음

- **사용목적** : 회원가입 절차 제거, 비밀번호 저장 불필요, 구글 간편 로그인 제공

- **JWT (JSON Web Token)**

JWT(JSON Web Token)는 **사용자의 인증 정보와 권한 정보를 포함하는 토큰** 기반 인증 방식이며 **토큰은 Header, Payload, Signature 세 부분으로 구성**되며 서버는 이를 검증하여 **사용자의 인증 상태를 확인**

- **사용목적** :

- Google OAuth는 사용자 인증만 담당 (OAuth Access Token은 수명이 짧음) → 사용자 신원 확인(Authentication)
 - 서비스 내부 권한(Role) 관리는 OAuth 인증 이후 FastAPI 서버에서 자체 **JWT(Access Token / Refresh Token)**를 발급하여 **API 인증과 권한 관리**를 수행 → 서비스 내부 인증(Authentication) 및 권한 관리 (Authorization)
- **Uvicorn**
FastAPI는 웹 프레임워크일 뿐, Spring Boot도 내부적으로 Tomcat이 실행해주는 것처럼 FastAPI도 실행해주는 **Uvicorn(ASGI Server)** 서버가 필요

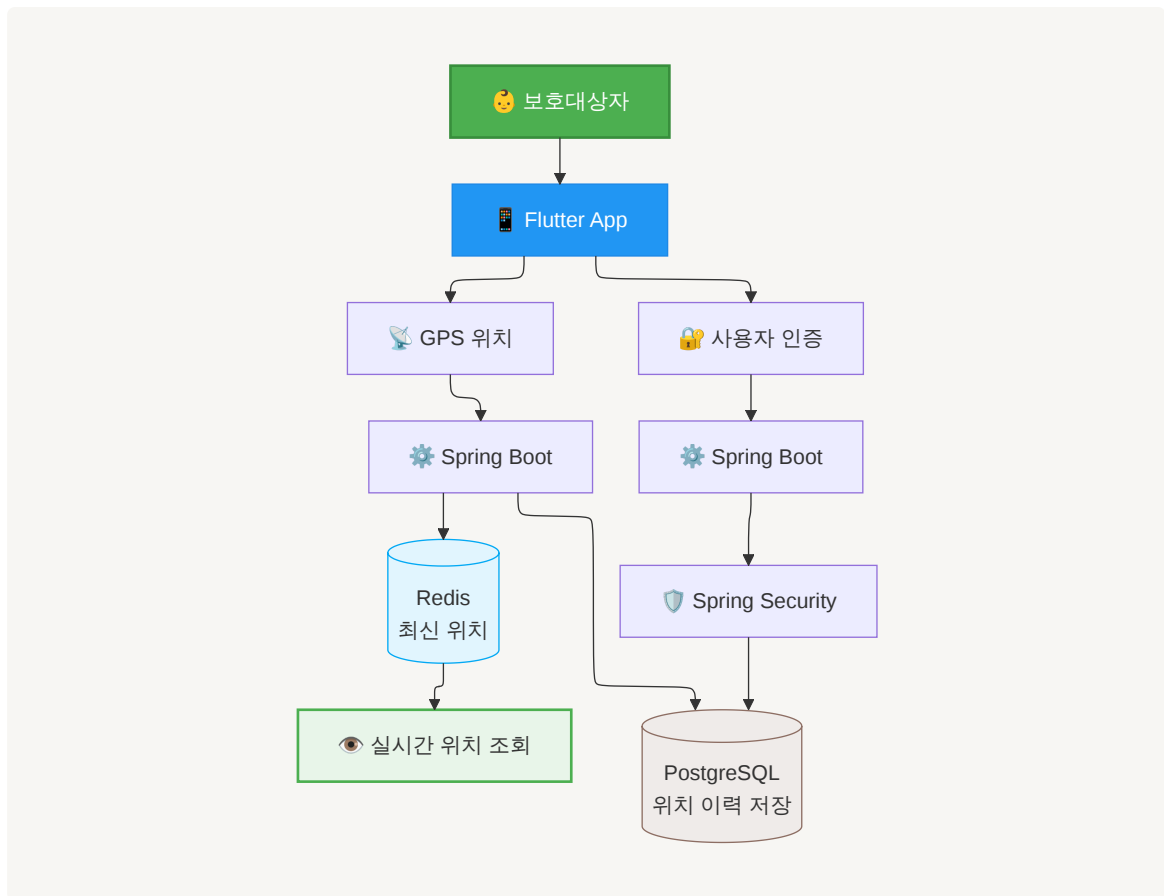
Flutter → HTTP / WebSocket → Uvicorn (ASGI Server) → FastAPI
→ postgreSQL

- **사용목적**
 - FastAPI 프레임워크를 실행하여, REST API와 WebSocket 기반의 비동기 요청을 효율적으로 처리하기 위함
- **Pandas**
데이터 분석 및 전처리 라이브러리로 머신러닝 모델을 학습시키기 전에 데이터를 가공하는 역할
 - 사용목적 : LocationHistory에는 GPS 원본 데이터만 저장되어 Pandas를 통해 데이터를 가공(전처리) 하여 Feature 생성하기 위함
- **XGBoost / LightGBM** : Feature Engineering을 통해 생성된 x_train과 y_train을 가지고 실제로 학습 시키는 데 사용되는 모델
- **Scikit-learn** : 머신러닝 작업을 도와주는 도구 모음,

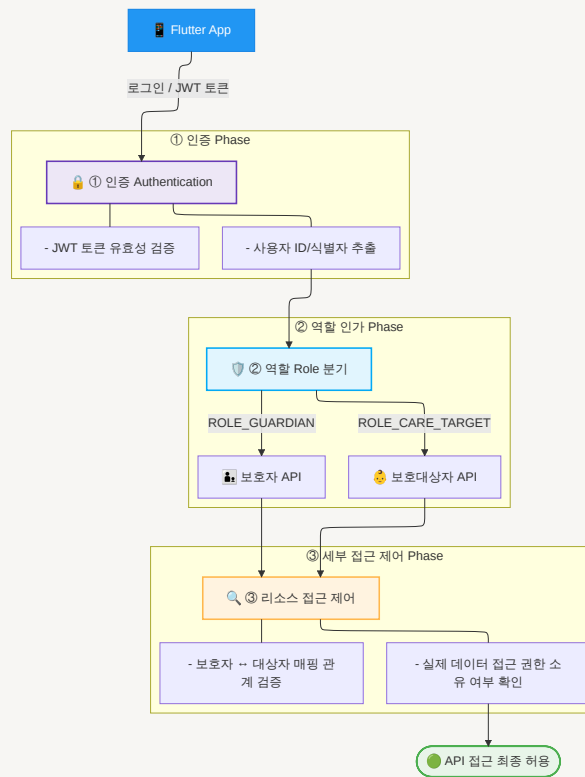
아키텍처

▼ 시스템 아키텍처 이미지

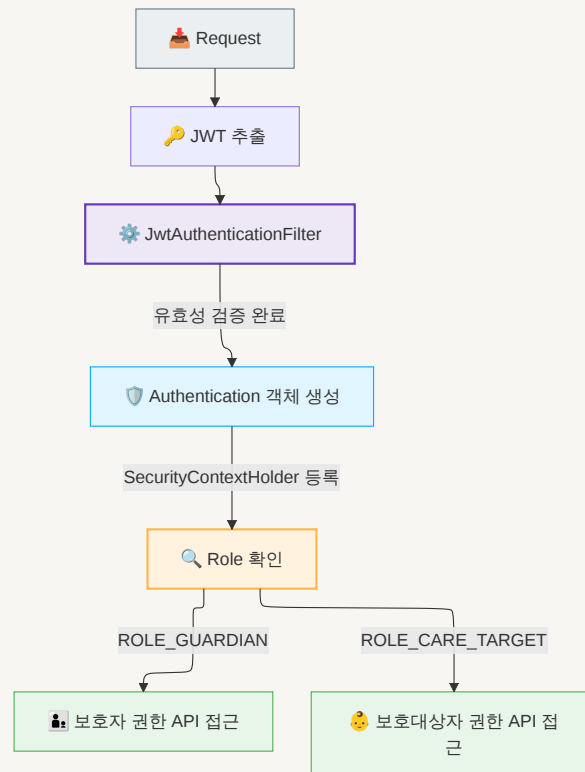
전체 어플리케이션



권한분기 구조도



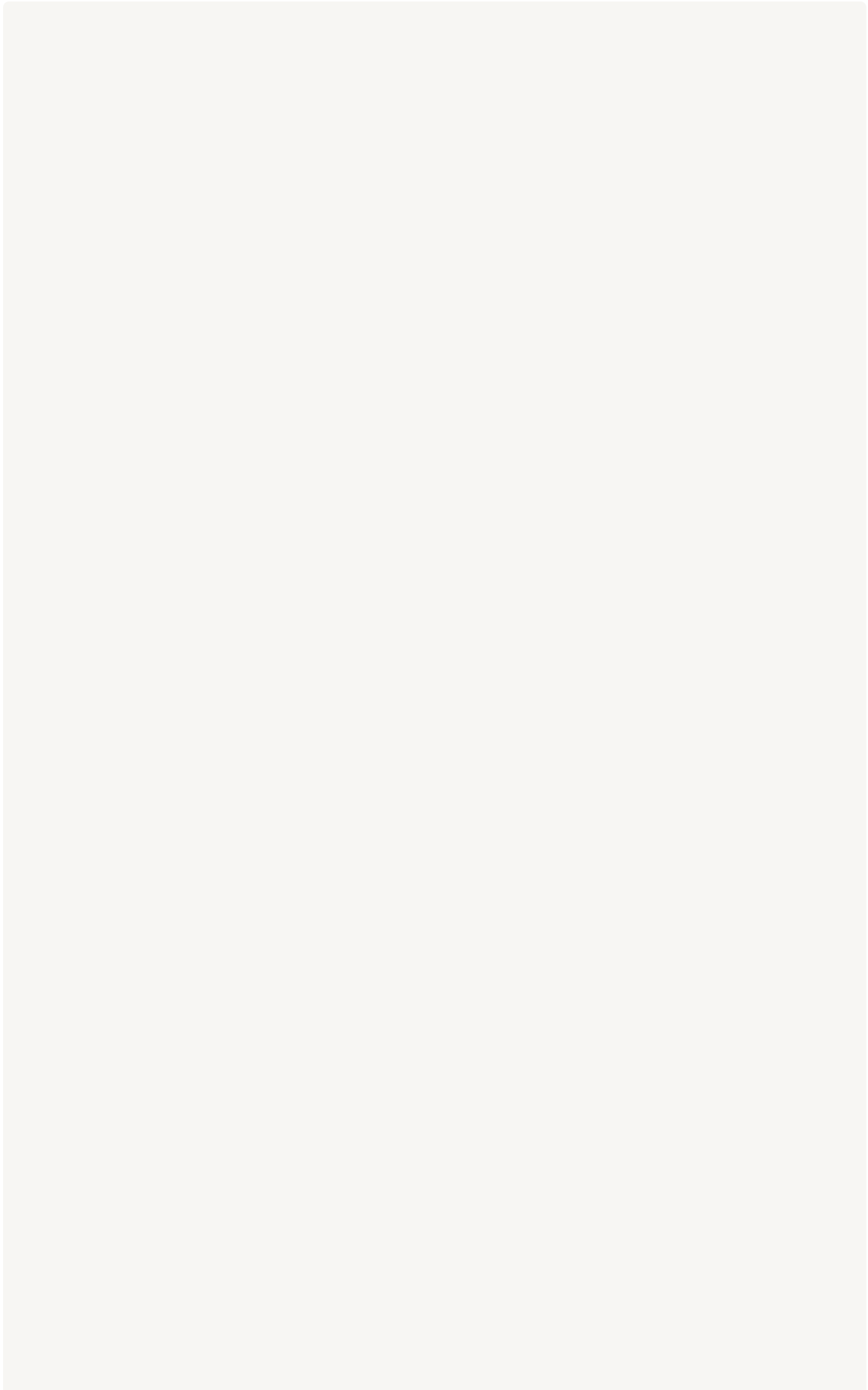
• 세부 권한 비교 흐름

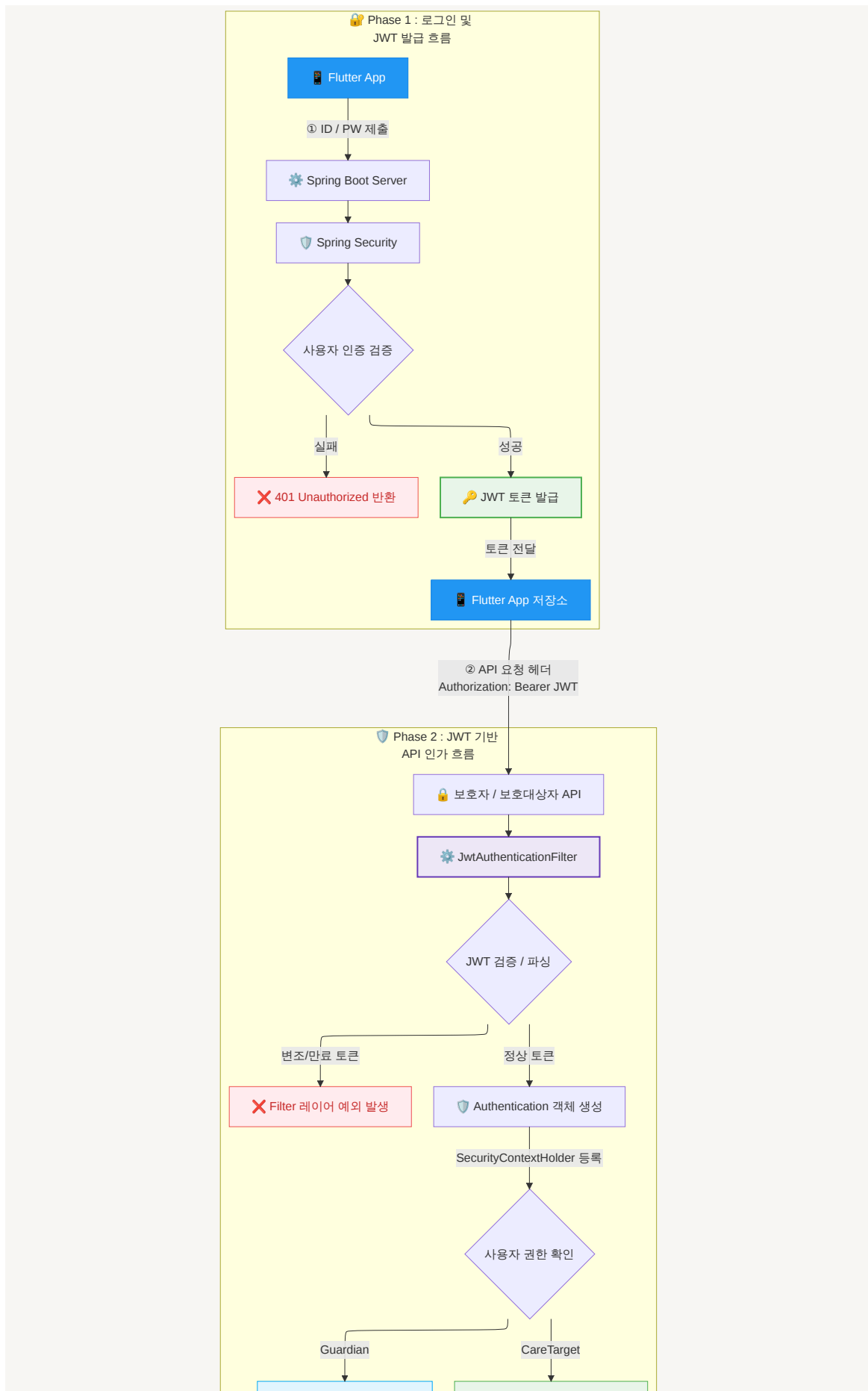


인증흐름 구조도

Spring Security + JWT 기반 RBAC + 리소스 접근 제어 구조

- 전체

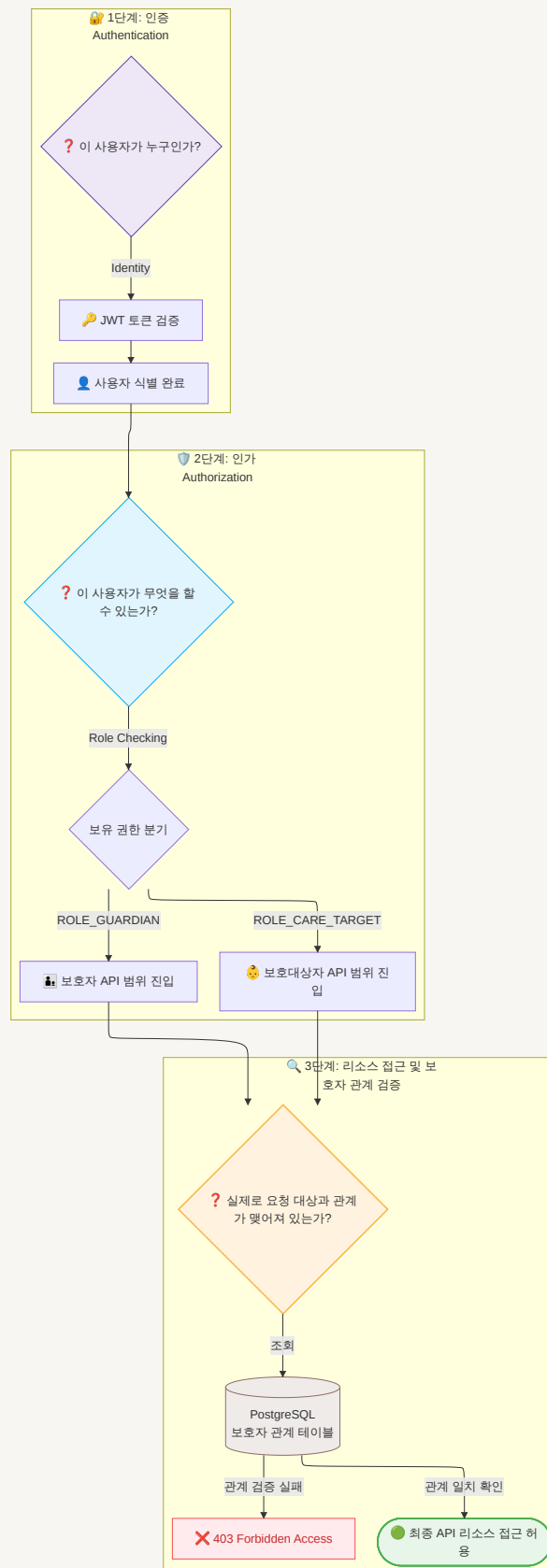




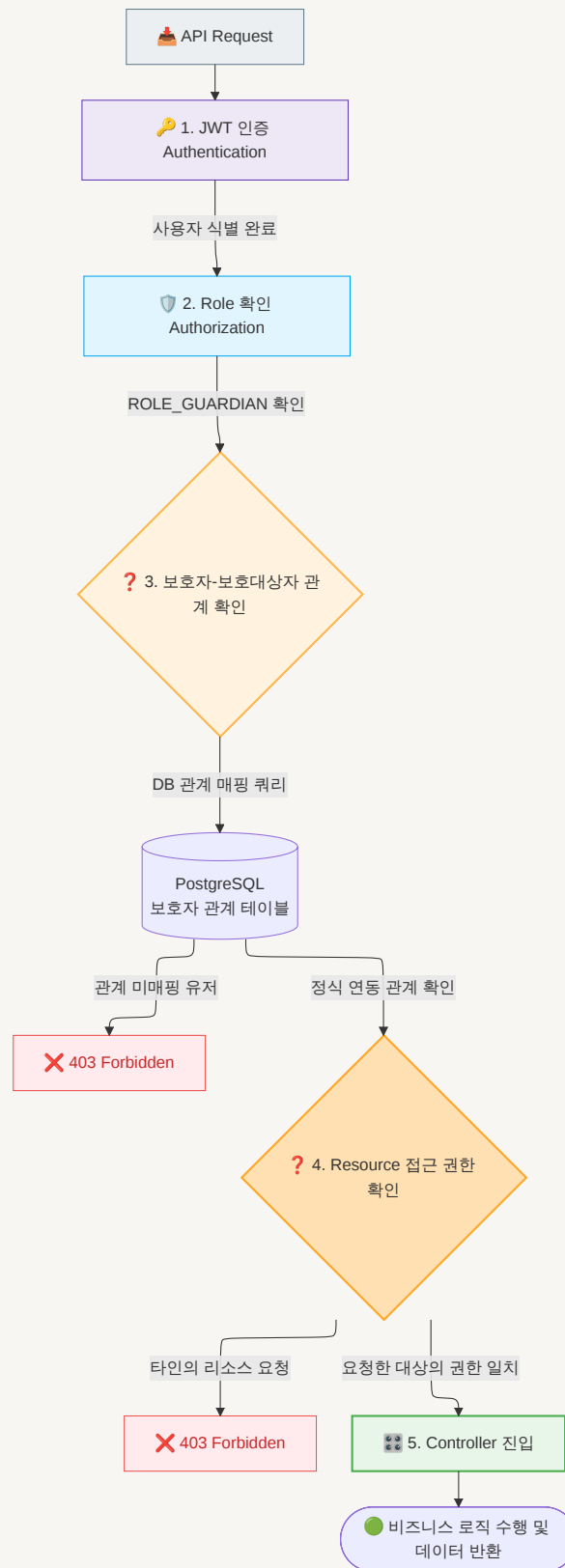
🔒 보호자 전용 API 매핑

👤 보호대상자 전용 API 매핑

- 인증과 인가



• 보호자 확인



🌸 역할 분리

- Spring boot Server

- ▼ 상세

Spring Boot Server

- ├── OAuth2 (Google) 로그인
- ├── JWT 발급 및 검증
- ├── Spring Security 인증/인가
- ├── GeoFence 진입 판단
- ├── 위치 정보 관리
- ├── 장소 관리 API
- ├── 보호자 관리 API
- ├── 방문 이력 관리
- ├── 알림 처리
- ├── FCM Push 알림
- └── WebSocket 실시간 위치 전송

- AI Server (ML / Fast API / LLM)

- ▼ 상세

AI Server (ML / FastAPI / LLM)

- |
- ├── 😊 ML Prediction
- | ├── Pandas 전처리
- | ├── NumPy 데이터 처리
- | ├── Feature Engineering
- | ├── Scikit-learn
- | ├── XGBoost
- | ├── LightGBM
- | ├── model.pkl
- | ├── ML 모델 추론
- | └── 방문 장소 예측
- |

- └─ ⚡ FastAPI
 - └─ AI 방문 장소 예측 API
 - └─ Prediction API
 - └─ AI Care Chat API
- └─ 🤖 LLM API
 - └─ LLM API 연동
 - └─ Prompt Engine
 - └─ 이동 기록 요약
 - └─ AI 리포트 생성
 - └─ 자연어 질의응답
 - └─ 이상행동 설명

• Redis

▼ 상세

Redis (빠른 임시 저장 / Cache)

- └─ 최신 위치 → CareTarget별 최신 위치 (빠른 실시간 조회 Redis 사용)
- └─ Refresh Token → TTL 설정
- └─ JWT Blacklist
- └─ FCM Token
- └─ AI Prediction Cache → 예측 결과 임시 저장
- └─ Chat / LLM Cache → AI 응답 임시 저장
- └─ GeoFence Cache → 보호대상자별 Geofence 정보
- └─ User Cache → 자주 조회되는 사용자 정보

• PostgreSQL

▼ 상세

PostgreSQL (영구 저장)

```
|
|— User
|— Guardian
|— CareTarget
|— Place
|— LocationHistory
|— ArrivalHistory
|— NotificationHistory
|— PredictionHistory
|— ChatHistory
|— pgvector
    |— AI Vector Embedding
```

🌸 REST API 명세서

▼ 상세

• 인증(Auth)

Method	URI	설명	권한
POST	/api/auth/oauth/login	OAuth 로그인	All
POST	/api/auth/logout	로그아웃	All
POST	/api/auth/refresh	AccessToken 재발급	All
GET	/api/auth/me	현재 사용자 정보	All

• 보호자(Guardian) API

◦ 보호 대상자 관리

Method	URI	설명
GET	/api/guardian/care-targets	보호 대상자 목록
POST	/api/guardian/care-targets	보호 대상자 등록
GET	/api/guardian/care-targets/{id}	상세 조회
PUT	/api/guardian/care-targets/{id}	수정

Method	URI	설명
DELETE	/api/guardian/care-targets/{id}	삭제

◦ 장소관리

Method	URI	설명
GET	/api/guardian/places	장소 목록
POST	/api/guardian/places	장소 등록
PUT	/api/guardian/places/{id}	수정
DELETE	/api/guardian/places/{id}	삭제

◦ 실시간 위치 조회

Method	URI	설명
GET	/api/guardian/location/current	현재 위치
GET	/api/guardian/location/history	이동 히스토리
WebSocket	/ws/guardian/location	실시간 위치 수신

◦ 방문 히스토리

Method	URI	설명
GET	/api/guardian/history/today	오늘 이동
GET	/api/guardian/history/date	날짜별 조회
GET	/api/guardian/history/place	장소별 조회

◦ AI 방문 예측 (머신러닝)

Method	URI	설명
GET	/api/guardian/ai/predict	오늘 방문 예상
GET	/api/guardian/ai/report	AI 리포트
GET	/api/guardian/ai/history	예측 이력

◦ AI 케어 비서 (LLM)

Method	URI	설명
POST	/api/guardian/ai/chat	AI 대화
POST	/api/guardian/ai/summary	이동 요약
POST	/api/guardian/ai/report	주간 리포트

Method	URI	설명
POST	/api/guardian/ai/explain	이상행동 설명
POST	/api/guardian/ai/search	자연어 이동기록 검색

- **알림**

Method	URI	설명
GET	/api/guardian/notifications	알림 목록
PUT	/api/guardian/notifications/{id}/read	읽음 처리
GET	/api/guardian/notifications/history	알림 이력

- **내정보**

Method	URI	설명
GET	/api/guardian/profile	내 정보
PUT	/api/guardian/profile	수정
PUT	/api/guardian/profile/image	프로필 변경

- **보호자(Guardian) API**

- **현재 위치**

Method	URI	설명
POST	/api/care-target/location	위치 전송
GET	/api/care-target/location	현재 위치
WebSocket	/ws/care-target/location	실시간 위치 송신

- **도착 확인**

Method	URI	설명
POST	/api/care-target/arrival/check	도착 확인
GET	/api/care-target/arrival/history	도착 기록

- **현재 위치 공유**

Method	URI	설명
POST	/api/care-target/share/location	현재 위치 공유

- **긴급 연락**

Method	URI	설명
POST	/api/care-target/emergency/call	보호자 전화
POST	/api/care-target/emergency/message	긴급 문자
POST	/api/care-target/emergency/location	현재 위치 전송

◦ AI 도우미

Method	URI	설명
POST	/api/care-target/ai/chat	AI 대화
POST	/api/care-target/ai/help	도움 요청
POST	/api/care-target/ai/navigation	등록 장소 안내
POST	/api/care-target/ai/location	현재 위치 설명

◦ 알림

Method	URI	설명
GET	/api/care-target/notifications	알림 조회
PUT	/api/care-target/notifications/{id}/read	읽음 처리

◦ 내 정보

Method	URI	설명
GET	/api/care-target/profile	내 정보
PUT	/api/care-target/profile	내 정보 수정

• 공통 내부 API (시스템)

서버 내부에서 사용하는 API (사용자 호출 x)

Method	URI	설명
POST	/internal/geofence/check	GeoFence 진입 판단
POST	/internal/fcm/send	FCM Push 발송
POST	/internal/ai/predict	머신러닝 예측 요청
POST	/internal/llm/chat	LLM 질의
POST	/internal/notification/send	알림 생성
POST	/internal/sms/send	SMS 발송

★ 보안

★ 권한 정책(Role-Based Authorization)

"UI 메뉴 분리" 와 "API 접근 제한"

- OAuth 로그인 후 JWT에 포함된 Role(Guardian/CareTarget)을 기준으로 사용자 화면(UI)을 동적으로 분리
- 서버에서는 RBAC(Role-Based Access Control)를 적용하여 Role별 API 접근을 제어
- UI에서 메뉴를 숨기는 것뿐만 아니라 **Spring Security**에서 서버 API 접근 자체를 권한 별로 차단

▼ 권한 분기표

기능	Guardian	CareTarget
OAuth 로그인	✓	✓
실시간 위치 조회	✓	✗
실시간 위치 전송	✗	✓
보호 대상자 관리	✓	✗
장소 관리	✓	✗
방문 히스토리 조회	✓	✗
AI 방문 예측	✓	✗
AI 케어 비서	✓	✗
AI 도우미	✗	✓
도착 확인	✗	✓
현재 위치 공유	✗	✓
긴급 연락	✗	✓
알림 조회	✓	✓
내 정보	✓	✓

▼ 권한 분기 흐름

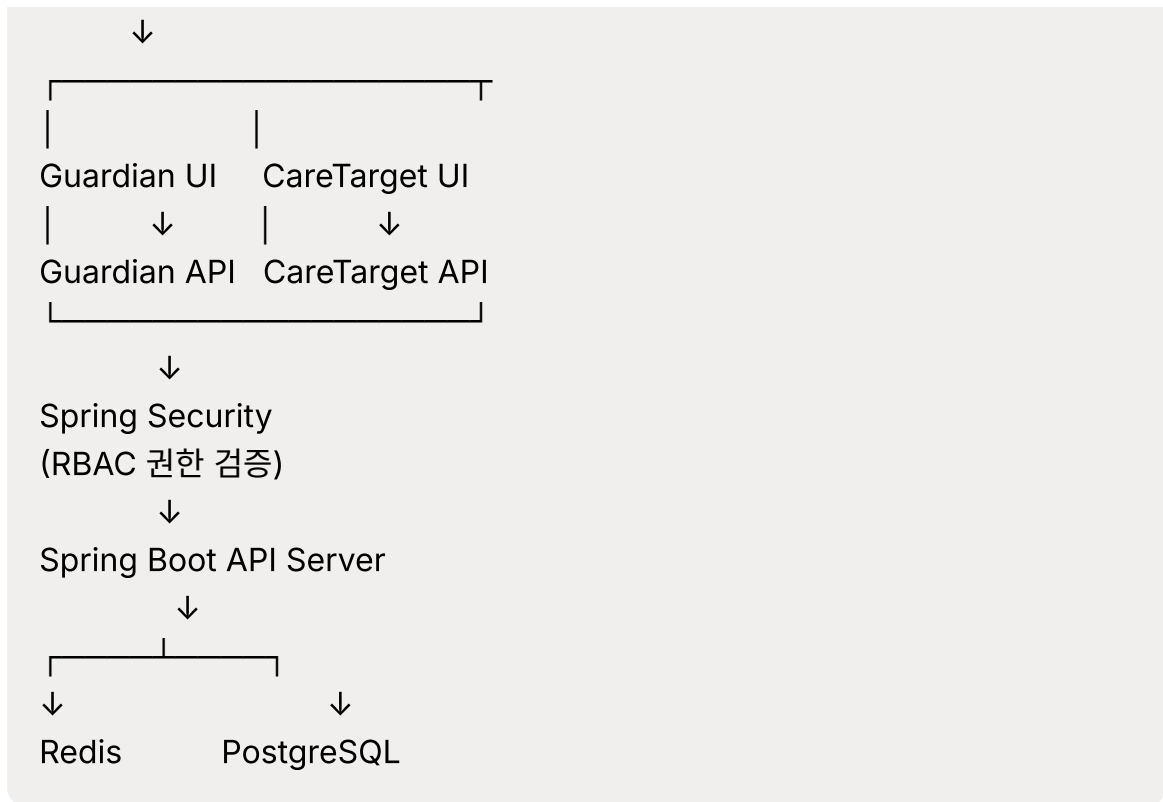
OAuth 로그인



JWT Access Token 발급



JWT Role 확인



★ 권한정리 (Role)

• 보호자(Guardian)

▼ 권한과 역할

- 역할 : 보호대상자의 상태를 **확인하고 관리하는 역할**
 - 실시간 위치 **조회**
 - 보호대상자 관리
 - 장소 관리
 - 방문 기록 조회
 - AI 방문 예측
 - AI 케어 비서

• 보호 대상자 (CareTarget)

▼ 권한과 역할

- 역할 : 자신의 위치와 상태를 **전송하고 보호자와 공유하는 역할**
 - 실시간 위치 **전송**

- 현재 위치 공유
- 도착 확인
- 긴급 연락
- AI 도우미

- 관리자: Admin

▼ 권한과 역할

- 시스템 관리자 Role
- 사용자 및 서비스 관리 권한

★캐시 전략

- **REST Cache (브라우저 / 클라이언트)**

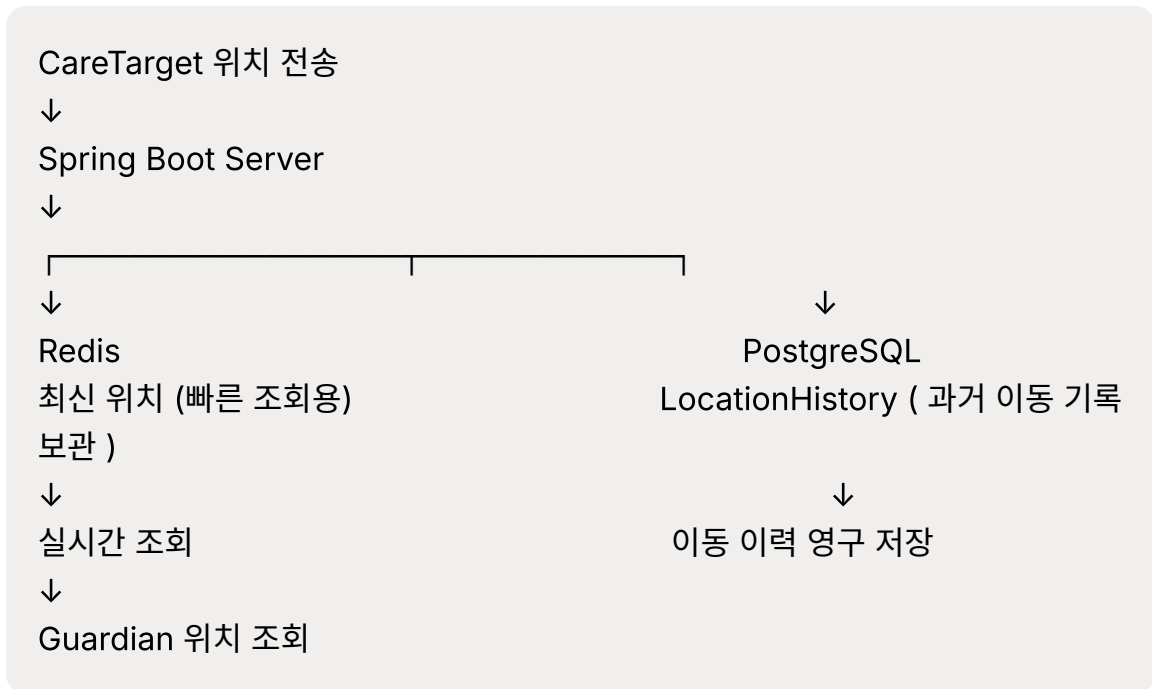
- 변경이 거의 없는 데이터의 반복 조회 시 활용 → 빠른 응답
- 서버 요청 및 네트워크 비용 감소

- **Redis Cache (서버)**

- **장소 조회** : CareTarget별 최신 위치를 저장하여 실시간 위치 조회 성능 향상
- **Place / GeoFence 정보** : 자주 변경되지 않는 장소 및 Geofence 정보를 캐싱하여 DB 조회 감소
- **CareTarget 정보** : 보호대상자 목록 및 기본 정보를 캐싱하여 반복 조회 성능 향상
- **AI 방문 예측 결과** : AI 모델을 반복 실행을 방지하여, 응답 속도 향상 및 AI 서버 부하 감소 (비용 감소)
- **AI 케어 비서 / LLM 응답** : 반복적인 요청에 대한 응답을 캐싱하여 LLM API 호출 및 비용 절감
- **Google Places API 조회 결과** : 동일 장소에 대한 외부 API 반복 호출을 줄여 API 호출 횟수 및 비용 절감
- **FCM Token** : 사용자별 FCM Token을 빠르게 조회하여 푸시 알림 발송
- **OAuth 사용자 정보** : OAuth 인증 후 사용자 정보를 임시 캐싱하여 반복적인 사용자 정보 조회 감소
- **Refresh Token** : TTL을 적용하여 만료 기간을 관리

- **JWT Blacklist** : 로그아웃 또는 토큰 무효화 시 Access Token을 TTL 기반으로 관리

- **시스템 동작 흐름**

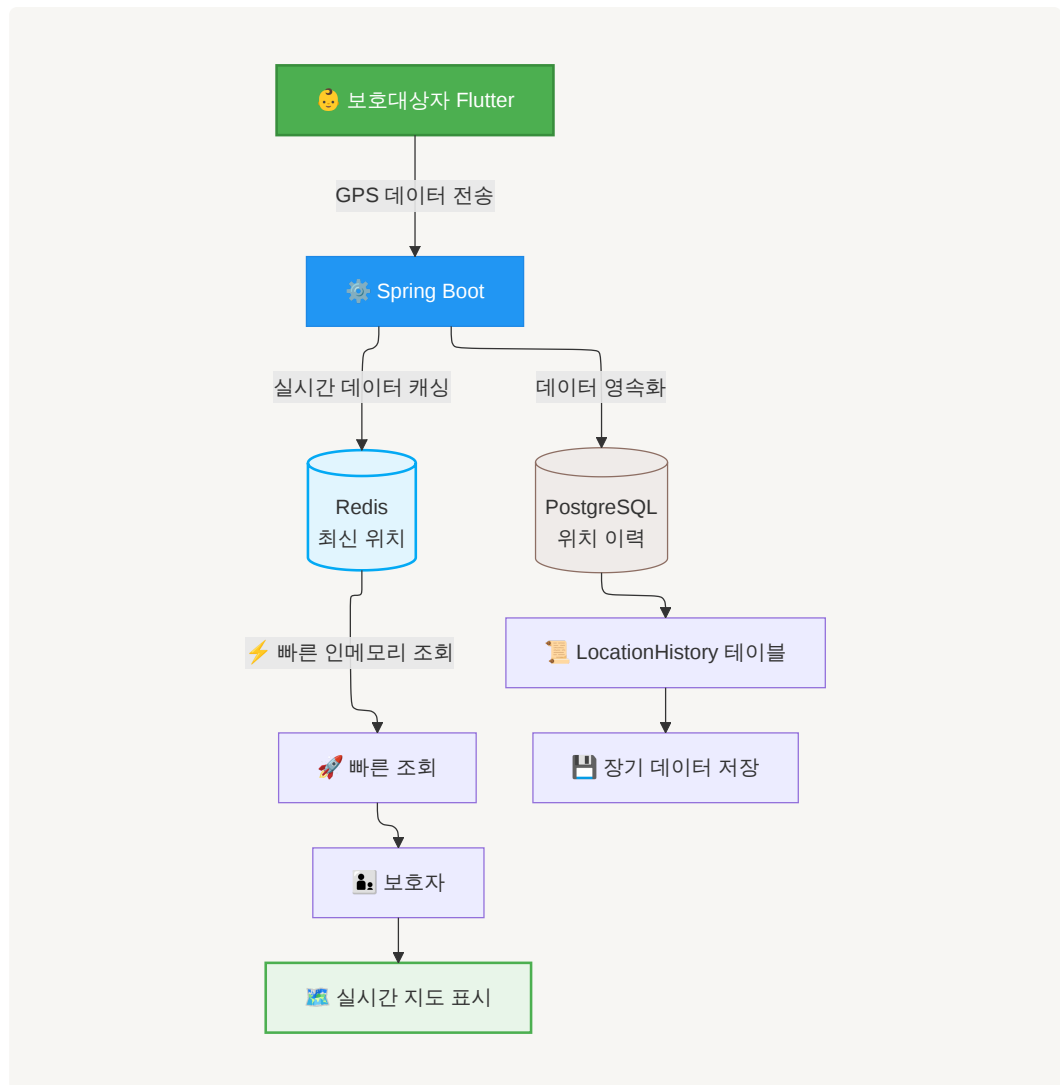


기대 효과

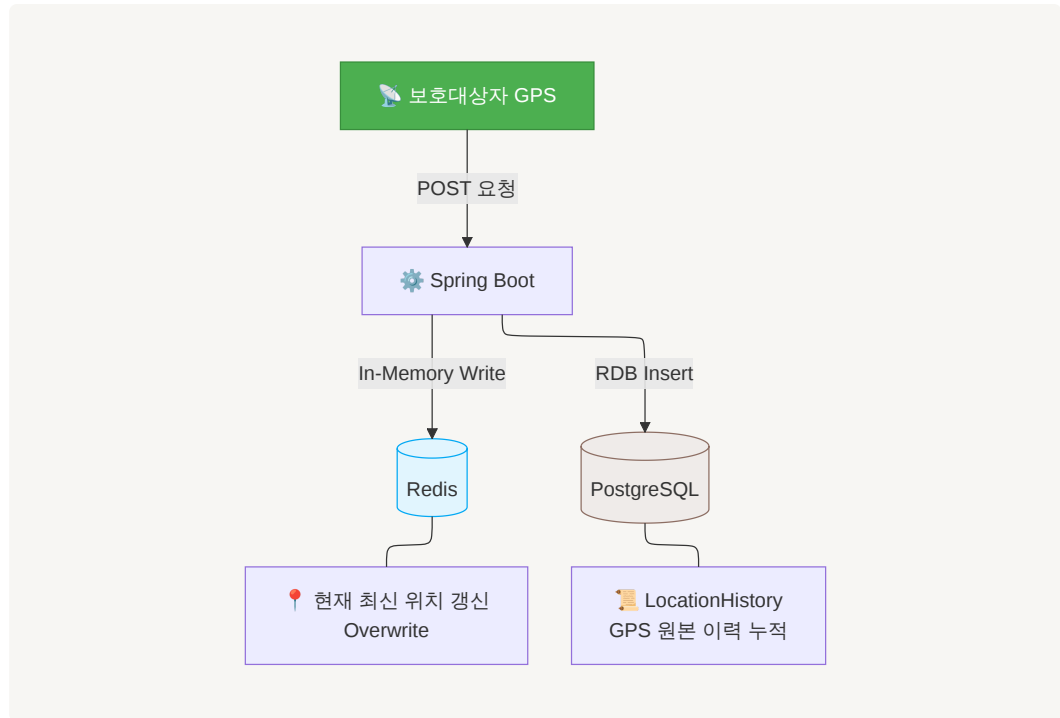
- 최신 위치, 장소 및 Geofence, 사용자 정보 등을 캐싱하여 **PostgreSQL 조회 횟수 감소**
- AI 방문 예측 및 LLM 응답을 캐싱하여 **AI 서버 및 외부 API 호출 최소화**
- Google Places API 결과를 캐싱하여 **외부 API 호출 횟수 및 비용 절감**
- FCM Token, Refresh Token, JWT Blacklist 등을 Redis에서 관리하여 **인증 및 알림 처리 성능 향상**
- TTL을 활용하여 **불필요한 캐시 데이터의 자동 만료 및 메모리 관리**

▼ 캐시 전략 구조도

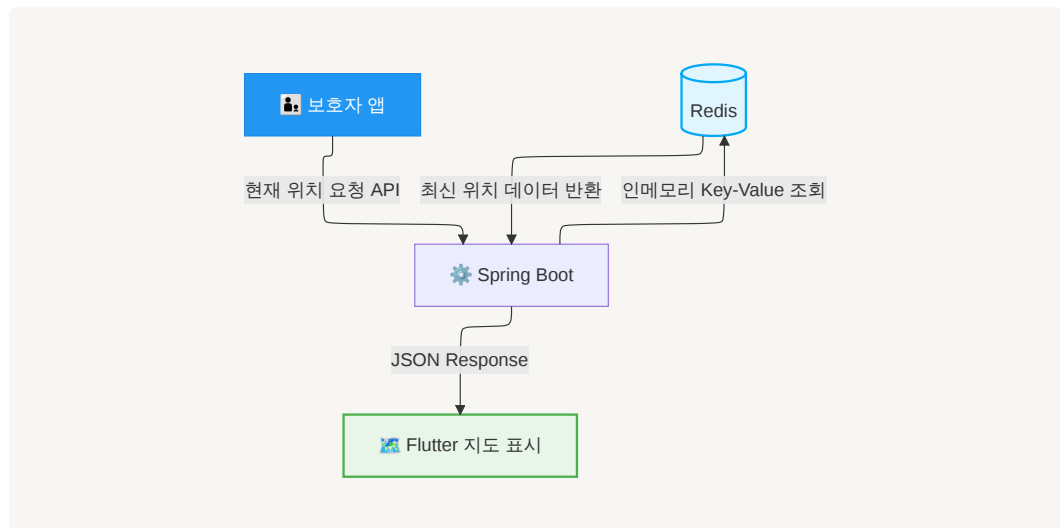
- 전체



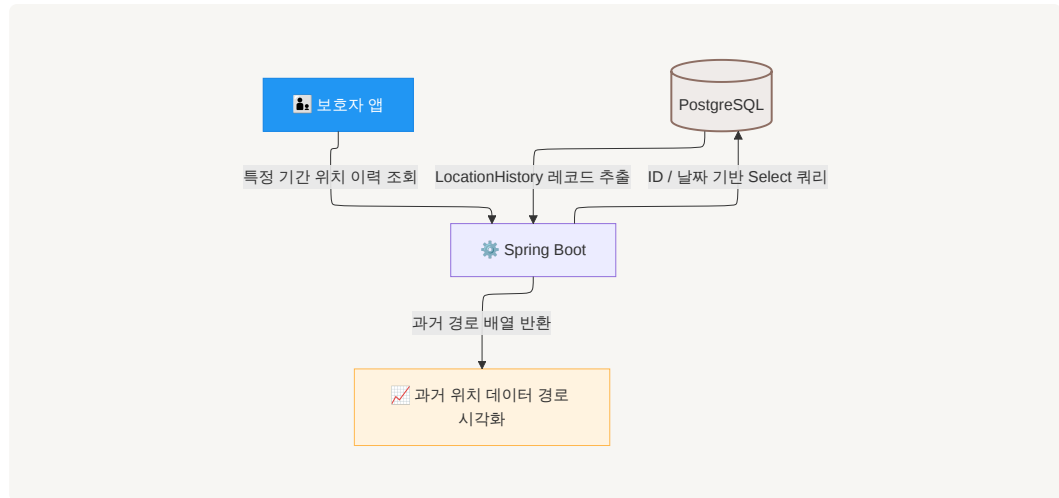
- 위치저장



• 조회 흐름



• 위치 이력 조회



🌸 기능 목록

1. 로그인

Google OAuth 2.0 + 자체 JWT 인증

▼ 사용 이유

- Google OAuth 2.0을 이용하여 회원가입 및 비밀번호 관리 부담 감소
- Google 계정으로 간편 로그인 제공
- OAuth 인증 성공 후 Spring Boot 서버에서 사용자 정보를 확인
- 서비스 자체 Access Token / Refresh Token 발급
- JWT에 사용자 ID와 Role(Guardian / CareTarget)을 포함하여 Spring Security에서 인증 및 권한 관리

2. Guardian 기능

- 실시간 위치
- 보호 대상자 관리
- 장소 관리
- 방문 히스토리
- AI 방문 예측
- AI 케어 비서
- 내 정보

3. CareTarget 기능

- 도착 확인
- 긴급 연락
- 현재 위치 공유
- AI 도우미
- 내 정보

4. 공통 기능

- 알림
- OAuth
- JWT
- WebSocket
- GeoFence
- Redis Cache

시나리오

• 공통 시나리오

▼ 로그인

- 사용자는 Google OAuth 간편 로그인을 통해 로그인한다.
- 서버는 OAuth 인증 정보를 검증한 후 자체 JWT(Access Token / Refresh Token)를 발급한다.
- 최초 로그인 시 사용자 역할(Role)을 선택한다.
 - Guardian(보호자)
 - CareTarget(보호 대상자)
- 선택된 Role 정보는 DB에 저장된다.
- 이후 로그인부터는 저장된 Role 정보를 기반으로 자동으로 화면을 분기한다.

★ Role 변경 정책 명시 (권한 잘못 설정했을 경우)

- 최초 로그인 시 Guardian / CareTarget Role을 선택한다.
- 선택한 Role은 이후 임의로 변경할 수 없다.

- Role 변경이 필요한 경우 서비스 운영 정책에 따라 별도 처리한다.

Spring Boot → WebSocket

▼ 알림

- 도착 알림
GeoFence 진입 → Guardian Push 발송
- 도착 확인 요청
GeoFence 진입 → CareTarget 확인 요청
- 재알림
5분 미응답 → 자동 재발송
- 긴급 알림
재알림 이후 미응답 → Guardian 긴급 알림
- AI 이상행동 알림
머신러닝 이상행동 감지 → Guardian Push 발송
- AI 방문 예측 알림
예측 결과 생성 → Guardian에게 오늘 예상 방문 장소 안내
- AI 주간 리포트 알림
Scheduler 실행 → LLM 리포트 생성 → Guardian Push 발송
- **알림 이력 관리**
모든 알림은 NotificationHistory에 저장

저장 항목

- 알림 발송 시간
- 읽은 시간
- 확인 시간
- 재발송 여부
- 알림 타입
- 응답 여부

- 사용자 ID
- 보호 대상자 ID

→ 관리자는 알림 이력을 통해 서비스 이용 현황과 사용자 행동을 추적할 수 있다.

▼ 내 정보

보호자 → 내 정보 메뉴 선택 → 조회가능

- 내 정보 조회
 - 이메일
 - 전화번호
 - OAuth 계정
 - 프로필 이미지
- 내 정보 수정
 - 이메일 수정
 - 전화번호 수정
 - OAuth 계정 조회
 - 프로필 이미지 변경

• Guardian 시나리오

- 지도 (Google Maps)
- 실시간 위치추적

▼ 상세

보호자 → 실시간 위치 메뉴 선택 → Spring Boot WebSocket 연결 → Redis에서 CareTarget 최신 위치 조회 → Guardian에게 실시간 위치 전달 → Google Maps 에 실시간 표시

Guardian
↓
Spring Boot WebSocket

↓
Redis
↓
CareTarget 최신 위치
↓
Guardian Google Maps

◦ 보호 대상자 관리

▼ 보호자 → 보호 대상자 목록

• 보호 대상자 목록 조회

- 목록 출력 : 이름 / 전화번호 / 관계 / 등록
- 상세 선택 → 상세 출력 :
 - 이름 / 전화번호 / 관계 / 이메일
 - 최근위치
 - 방문 히스토리
 - AI 방문 예측
 - AI 리포트
 - 수정 / 삭제

• 보호 대상자 등록

• 보호 대상자 수정

• 보호 대상자 삭제

◦ 장소 관리

▼ 보호자 → 장소 검색

• 장소 목록 : 장소 명칭과 일부 정보 / 방문 목적

• 장소 등록

- Google Places API / Kakao Local API / Naver Local API 중 사용

→ 검색 결과 출력 → 장소 선택 → **GeoFence 반경 설정** → 등록 완료 (**GeoFence 기준 장소로 사용**)

- 장소 수정

- Google Places API / Kakao Local API / Naver Local API 중 사용 → 장소 재검색 → 장소 선택 → **GeoFence 반경 설정** → 수정 완료 (**GeoFence 기준 장소로 사용**)

- 장소 삭제

- 방문 히스토리

- 보호자 → 방문 히스토리 메뉴 → LocationHistory 조회 → 이동경로 출력

- ▼ 이동경로 목록

- GPS 좌표를 일정 주기(예: 30초 또는 1분)로 저장
- UI 출력 예시

집 ● ————— ● 학교 ————— ● 학원
08:10 08:35 15:20

- 방문시간 / 체류시간/ 출발시간 / 조회 가능

- AI 방문 예측

- ▼ 상세

보호자 → AI 방문 예측 메뉴 → Prediction API 호출 → 머신러닝 모델 예측
→ 오늘 방문 예상 장소 / 다음 이동 장소 / 예상확률 출력

예시) ○○복지관 방문 확률 82% → ○○시장 방문 확률 50%

- AI 케어 비서

- ▼ 상세

보호자 → 자연어 질문 입력 → LLM API 호출 → PredictionHistory / LocationHistory / AI 결과 조회 → LLM 응답 생성 → 자연어 결과 출력

```

Guardian - 자연어 질문 입력
↓
Spring Boot ( LLM API 호출 )
↓
AI Care API 요청
↓
Spring Boot
|—— LocationHistory 조회
|—— VisitHistory 조회
|—— PredictionHistory 조회
↓
필요한 데이터 정리
↓
FastAPI
↓
LLM
↓
자연어 응답
↓
Spring Boot
↓
Guardian

```

▼ 비서 기능 예시 목록

- **자연어 질의응답** : 보호자가 질문하면 AI가 답변
- **오늘 예상 방문지 안내** : 머신러닝 예측 결과를 자연어로 설명
- **이동 기록 요약** : 하루/주간 이동 내역 자동 요약
- **이동 패턴 분석** : 최근 자주 방문하는 장소 분석
- **이상행동 설명** : 이상행동 발생 원인을 자연어 설명
- **자연어 방문 검색** : "지난주 병원 기록"처럼 검색 가능
- **AI 주간 리포트** : 이동거리·방문횟수·예측결과 자동 생성
- **보호자 상담** : 최근 이동 패턴 정상 여부 안내

◦ 알림

- 내 정보

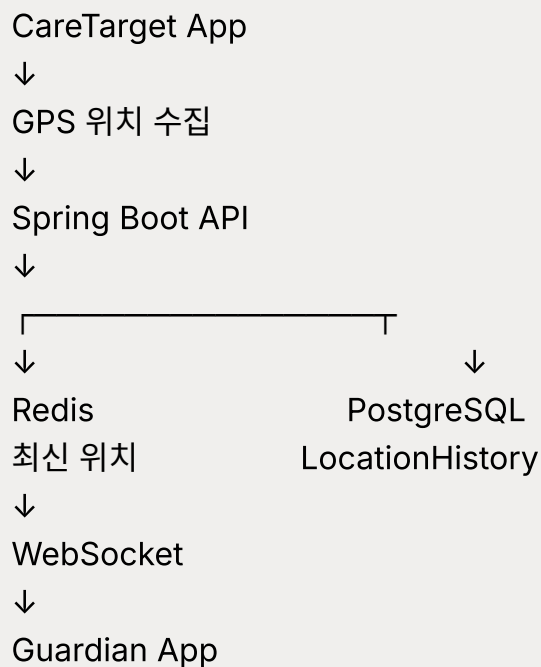
- CareTarget 시나리오

CareTarget 로그인 → CareTarget 전용 화면 진입 →

- 도착 확인
- 실시간 위치 공유(실시간 위치 전송)

- ▼ 상세

GPS 수집 → 30초 또는 1분 주기 → Spring Boot API 전송 →
LocationHistory 저장 → Guardian 지도 실시간 갱신



- GeoFence 도착 확인

- ▼ 상세

GeoFence 진입 → Spring Boot → Redis 최신 위치 저장 → GeoFence 진입 여부 판단 → 진입 감지 → Guardian "도착 알림" FCM 발송 →
CareTarget "도착 확인 버튼을 눌러주세요." 알림 / Guardian " xx 장소에 도착했습니다 " 알림

- **YES** → 도착 확인 완료 → Guardian에게 확인 완료 알림
- **응답 없음** → 5분 → 재알림 → 응답 없음 → Guardian 긴급 알림 발송

- 긴급 연락

- ▼ 상세

- 사용자 → 긴급 연락 버튼 → 긴급 상황에서 추가 인증 없이 즉시 접근 가능 → 메뉴 선택 (보호자 전화/문자/현재 위치 공유) → 확인 → 즉시 실행

- AI 도우미

- ▼ 상세

- 보호 대상자 → AI 도우미 실행 → 음성 또는 텍스트 입력 → LLM 호출 → 답변 제공

- 예시 목록

- 오늘 일정 안내
 - 등록 장소 안내
 - 보호자 연락 방법 안내
 - 앱 사용 도움말

- 알림

- 내 정보

- AI 시나리오

- Guardian 전용

▼ 방문 예측

LocationHistory (GPS 원본) → 체류시간 분석 → Google Places API
를 통한 장소명 변환 → VisitHistory 생성 (등록 장소 + 미등록 장소 모두
저장) → Feature Engineering → XGBoost / LightGBM →
Prediction API (예상 방문 장소 및 방문 확률 생성) → Guardian 화면
출력 → 자주 방문한 미등록 장소 발견 → ""자주 방문하는 장소입니다. 방
문 장소로 등록하시겠습니까? (GeoFence 장소)" → Guardian 승인 →
Place 테이블 등록

- **LocationHistory (GPS 원본)** : Care_Target이 이동한 모든 위치 저장
테이블
- **VisitHistory (방문 히스토리)** : GPS를 분석하여 10분이상 체류, 장소명
조회 (Google Places API), 방문 시작/종료 시간 계산 후 생성, 등록 장
소와 미등록 장소 모두 저장 → **AI 학습 대상 테이블**
- **등록 장소와 미등록 장소 모두 저장하는 이유** : 미등록 장소 중 자주 등록하
는 장소도 학습 해야하기 때문

• 학습 시나리오

VisitHistory → 요일 / 시간 / 장소 / 체류시간 / 방문횟수 → Feature
Engineering → XGBoost → 예측

- UI 등록 장소와 미등록 장소로 방문 히스토리 분류하여 출력
 - 등록장소 : GeoFence 기준 장소만 표시
 - 미등록 장소 : 보호자가 확인 가능 및 장소 추가 기

"자주 방문하는 장소입니다. GeoFence 장소로 등록하시겠습니
까?"

→ 등록 → Place 테이블 저장 → 다음부터 도착 알림 및
GeoFence 적용

▼ 이상행동 탐지

머신 러닝 → 평소 이동 패턴 분석 → 평소와 다른 이동 감지 → 이상행동 판단
→ Guardian에게 AI 이상행동 알림

▼ AI 케어 비서

Guardian 질문 → LLM → PredictionHistory / LocationHistory / 조회 →
자연어 생성 → 답변 제공

- 예시 목록
- 오늘 방문 예정 장소
- 이동 기록 요약
- 병원 방문 기록 조회
- 최근 이동 패턴 분석
- 이상행동 설명
- 주간 리포트 생성

▼ 시나리오 1 : 오늘 방문 예정 장소 조회

보호자
↓
AI 케어 비서
↓
"오늘 어디 방문 예정인가요?"
↓
머신러닝
↓
오늘 복지관 방문 확률 82%
↓
LLM
↓
"최근 이동 패턴을 분석한 결과
오늘 오전에는 ○○복지관을
방문할 가능성이 높습니다."

↓
보호자 화면 출력

▼ 시나리오 2 : 이동 기록 요약

보호자
↓
"오늘 이동기록 요약해줘"
↓
LocationHistory 조회
↓
LLM
↓
"오늘 오전 9시에 자택을 출발하여
10시에 ○○병원에 도착하였으며,
12시에 귀가하였습니다."
↓
출력

▼ 시나리오 3 : 이상행동 설명

머신러닝
↓
이상행동 감지
↓
LLM
↓
"평소 방문하지 않는 지역에
2시간 이상 머무른 것으로
판단되어 이상행동으로 분석되었습니다."
↓
보호자 알림

▼ 시나리오 4 : 자연어 검색

보호자
↓
"지난주 병원 방문 기록 보여줘"
↓
AI
↓
LocationHistory 검색
↓
LLM
↓
"지난주에는 총 2회 병원을 방문하였으며,
6월 11일 오전 9시와
6월 14일 오후 2시에 방문했습니다."

▼ 시나리오 5 : 주간 리포트

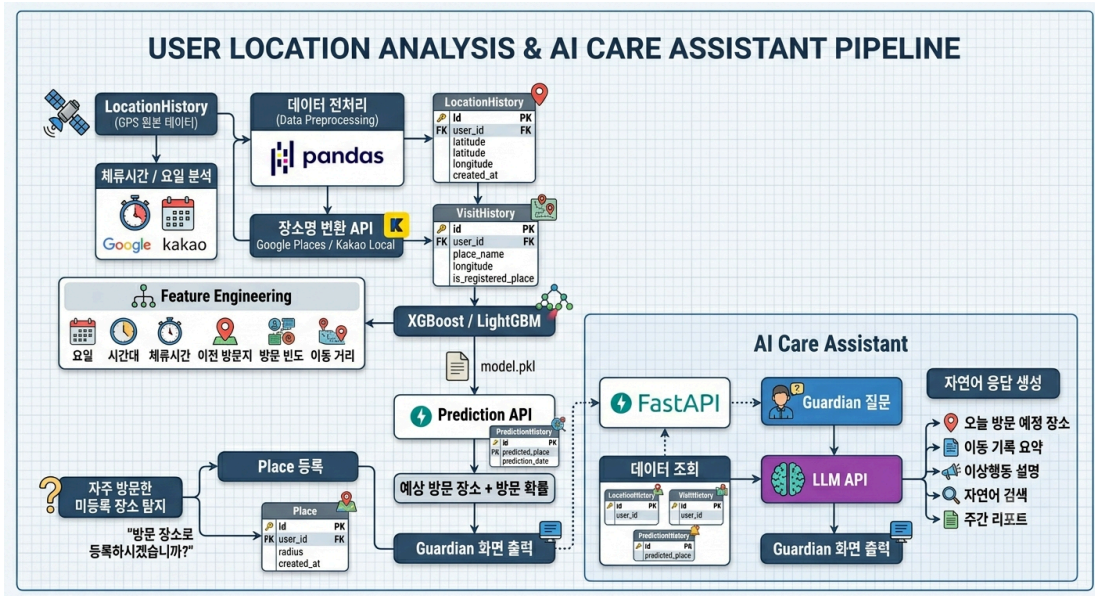
매주 일요
Spring Boot Scheduler
↓
PostgreSQL 조회
↓
LocationHistory
VisitHistory
PredictionHistory
↓
Spring Boot
↓
FastAPI
↓
LLM
↓
리포트 생성
↓
Spring Boot
↓

FCM
↓
Guardian

- CareTarget 전용
 - AI 도우미

AI 데이터 전처리 및 모델 학습

- 흐름 구조도
 - ▼ 참고이미지

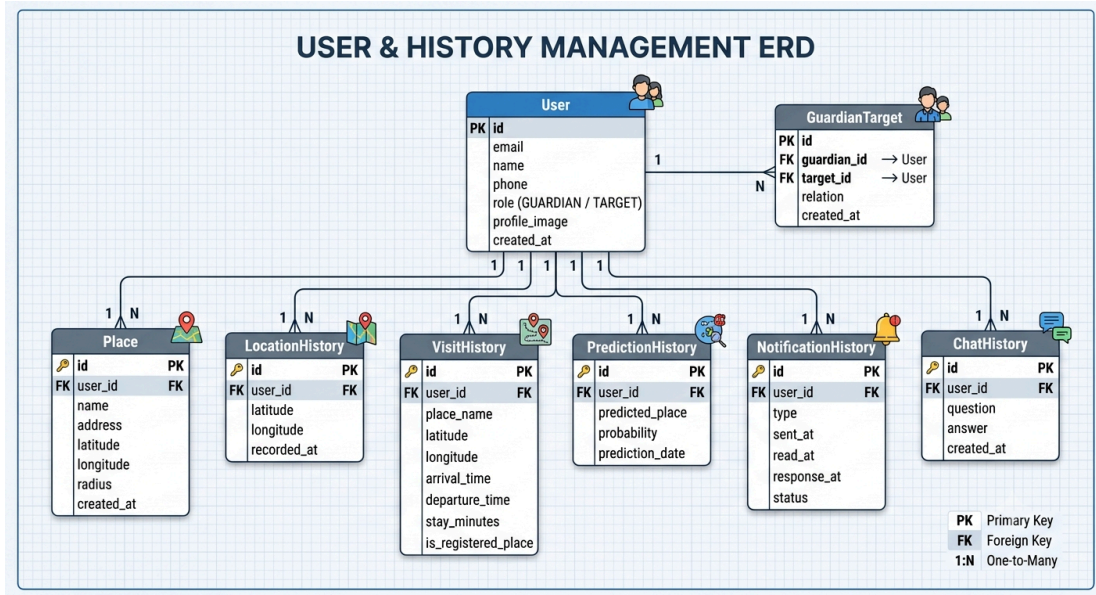


AI 파이프라인.png

Database

- ERD

▼ 이미지



DB_ERD.png

• 필수 엔터티

| 엔터티 | 테이블명 | 역할 |

| 사용자 | User | Guardian / CareTarget / Admin 사용자 정보 및 Role 관리 |

| 보호자-대상자 관계 | GuardianTarget | Guardian과 CareTarget 간 보호 관계 및 관계 정보 관리 |

| 장소 | Place | 보호 대상자의 등록 장소 및 GeoFence 기준 정보 관리 |

| 위치 이력 | LocationHistory | CareTarget의 GPS 원본 위치 및 이동 경로 데이터 저장 |

| 방문 이력 | VisitHistory | 위치 데이터를 분석하여 생성된 방문 장소, 시간, 체류 정보 저장 |

| AI 예측 이력 | PredictionHistory | AI 방문 예측 결과 및 예측 확률 저장 |

| 알림 이력 | NotificationHistory | FCM 알림 발송, 읽음 및 응답 이력 관리 |

| AI 대화 이력 | ChatHistory | AI 케어 비서의 질문 및 응답 대화 기록 저장 |

• 데이터 저장 흐름

- 위치 데이터 (GPS 원본 데이터를 영구 저장하는 흐름)

CareTarget App

↓

GPS 위치 수집

↓

Spring Boot

↓

JPA

↓

PostgreSQL

↓

LocationHistory

◦ **LocationHistory**

- CareTarget의 GPS 원본 위치 데이터 저장
- 위도 / 경도 / 위치 기록 / 기록 시간 저장
- 방문 분석 및 이동 경로 조회에 활용 → Guardian의 실시간 위치 조회 및 이동 경로 조회에 활용
- 이후 방문 데이터 생성의 원천 데이터로 활용

◦ 방문 데이터 (GPS 원본을 분석해서 방문 단위 데이터로 만드는 흐름)

LocationHistory

↓

GPS 위치 데이터 분석

↓

위치 변화 / 체류 시간 분석

↓

일정 시간 이상 동일 장소 체류

↓

방문 지점 판별

↓

장소 정보 조회

↓

Google Places API

↓
장소명 / 주소 확인
↓
등록 장소 여부 확인
↓
VisitHistory 생성
↓
PostgreSQL

○ VisitHistory

- GPS 원본 데이터를 기반으로 생성된 방문 단위 데이터
- 방문 장소 / 도착 시간(방문 시작 시간) / 이탈 시간(방문 종료 시간) / 체류 시간 / 방문 횟수 / 등록 장소 여 저장
- AI 방문 예측 학습 데이터로 활용

○ AI 방문 예측 데이터 (AI 학습/추론에 활용하고 예측 결과를 저장하는 흐름)

VisitHistory
↓
FastAPI
↓
데이터 전처리
↓
Feature Engineering
↓
요일 / 시간 / 장소 / 체류시간 / 방문횟수
↓
XGBoost / LightGBM
↓
방문 장소 및 방문 확률 예측
↓
PredictionHistory
↓
PostgreSQL

- **PredictionHistory**

- AI가 생성한 방문 예측 결과 저장
- 예상 방문 장소 / 확률 / 예측 날짜 저장
- Guardian의 AI 방문 예측 및 AI 케어 비서에서 활용

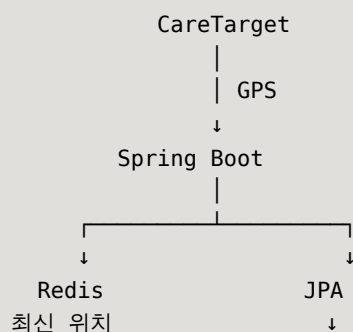
- **알림 데이터**

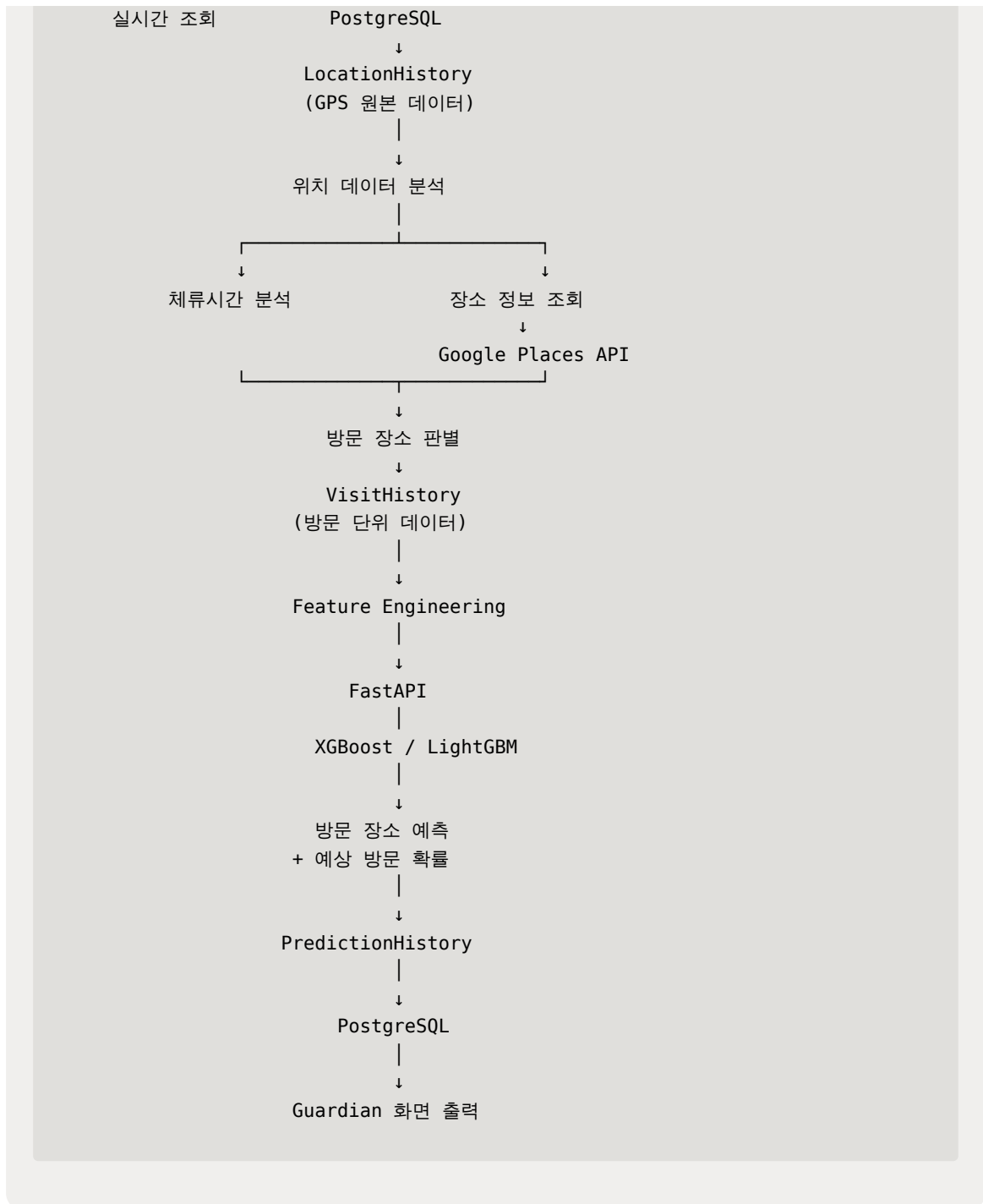
GeoFence / AI / 시스템 이벤트
↓
Spring Boot
↓
FCM
↓
사용자 알림
↓
NotificationHistory
↓
PostgreSQL

- **NotificationHistory**

- 알림 발송 및 사용자 응답 이력 저장
- 발송 시간 / 읽은 시간 / 응답 시간 / 알림 상태 관리

▼ 전체 데이터 흐름





- 테이블 생성 SQL

- ▼ SQL 문

```

-- 1. User 테이블
CREATE TABLE "User" (
  id SERIAL PRIMARY KEY,

```

```

email VARCHAR(255) UNIQUE NOT NULL,
name VARCHAR(100) NOT NULL,
phone VARCHAR(20),
role VARCHAR(20) NOT NULL CHECK (role IN ('ADMIN', 'GUARDIAN', 'CARE_TARGET')),
profile_image TEXT,
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

-- 2. GuardianTarget 테이블 (User 간의 다대다 관계 해소 엔티티)

```

CREATE TABLE "GuardianTarget" (
    id SERIAL PRIMARY KEY,
    guardian_id INT NOT NULL,
    target_id INT NOT NULL,
    relation VARCHAR(50),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,

```

-- 외래키 제약조건 (User 테이블 참조)

```

    CONSTRAINT fk_gt_guardian FOREIGN KEY (guardian_id) REFERENCES "User"(id) ON DELETE CASCADE,
    CONSTRAINT fk_gt_target FOREIGN KEY (target_id) REFERENCES "User"(id) ON DELETE CASCADE,

```

-- 보호자와 피보호자 쌍이 중복되지 않도록 유니크 제약 추가 (선택)

```

    CONSTRAINT uq_guardian_target UNIQUE (guardian_id, target_id)
);

```

-- 3. Place 테이블

```

CREATE TABLE "Place" (
    id SERIAL PRIMARY KEY,
    user_id INT NOT NULL,
    name VARCHAR(150) NOT NULL,
    address TEXT,

```

```

latitude NUMERIC(10, 7) NOT NULL,
longitude NUMERIC(11, 7) NOT NULL,
radius INT NOT NULL DEFAULT 100, -- 기본 반경(m) 설정 예시
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,

```

```

CONSTRAINT fk_place_user FOREIGN KEY (user_id) REFERENCES "User"(id) ON DELETE CASCADE
);

```

-- 4. LocationHistory 테이블 (GPS 원본 데이터)

```

CREATE TABLE "LocationHistory" (
    id BIGSERIAL PRIMARY KEY, -- 데이터가 대량으로 쌓일 수 있으므로 BIGSERIAL 권장
    user_id INT NOT NULL,
    latitude NUMERIC(10, 7) NOT NULL,
    longitude NUMERIC(11, 7) NOT NULL,
    recorded_at TIMESTAMP WITH TIME ZONE NOT NULL,

```

```

CONSTRAINT fk_lh_user FOREIGN KEY (user_id) REFERENCES "User"(id) ON DELETE CASCADE
);

```

-- 5. VisitHistory 테이블 (방문 이력)

```

CREATE TABLE "VisitHistory" (
    id BIGSERIAL PRIMARY KEY,
    user_id INT NOT NULL,
    place_name VARCHAR(150),
    latitude NUMERIC(10, 7) NOT NULL,
    longitude NUMERIC(11, 7) NOT NULL,
    arrival_time TIMESTAMP WITH TIME ZONE NOT NULL,
    departure_time TIMESTAMP WITH TIME ZONE,
    stay_minutes INT,
    is_registered_place BOOLEAN DEFAULT FALSE,

```

```

CONSTRAINT fk_vh_user FOREIGN KEY (user_id) REFERENCES "User"(id) ON DELETE CASCADE

```



```

);

-- 6. PredictionHistory 테이블 (AI 예측 기록)
CREATE TABLE "PredictionHistory" (
    id BIGSERIAL PRIMARY KEY,
    user_id INT NOT NULL,
    predicted_place VARCHAR(150) NOT NULL,
    probability NUMERIC(4, 3) NOT NULL, -- 예: 0.955
    (95.5%)
    prediction_date DATE NOT NULL,

    CONSTRAINT fk_ph_user FOREIGN KEY (user_id) REFER
ENCES "User"(id) ON DELETE CASCADE
);

-- 7. NotificationHistory 테이블 (알림 기록)
CREATE TABLE "NotificationHistory" (
    id BIGSERIAL PRIMARY KEY,
    user_id INT NOT NULL,
    type VARCHAR(50) NOT NULL, -- 알림 종류 (예: 이상동
행, 미등록장소 탐지 등)
    sent_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_
TIMESTAMP,
    read_at TIMESTAMP WITH TIME ZONE,
    response_at TIMESTAMP WITH TIME ZONE,
    status VARCHAR(20) DEFAULT 'SENT', -- SENT, READ,
RESPONDED 등

    CONSTRAINT fk_nh_user FOREIGN KEY (user_id) REFER
ENCES "User"(id) ON DELETE CASCADE
);

-- 8. ChatHistory 테이블 (AI 어시스턴트 대화 이력)
CREATE TABLE "ChatHistory" (
    id BIGSERIAL PRIMARY KEY,
    user_id INT NOT NULL,
    question TEXT NOT NULL,
    answer TEXT NOT NULL,

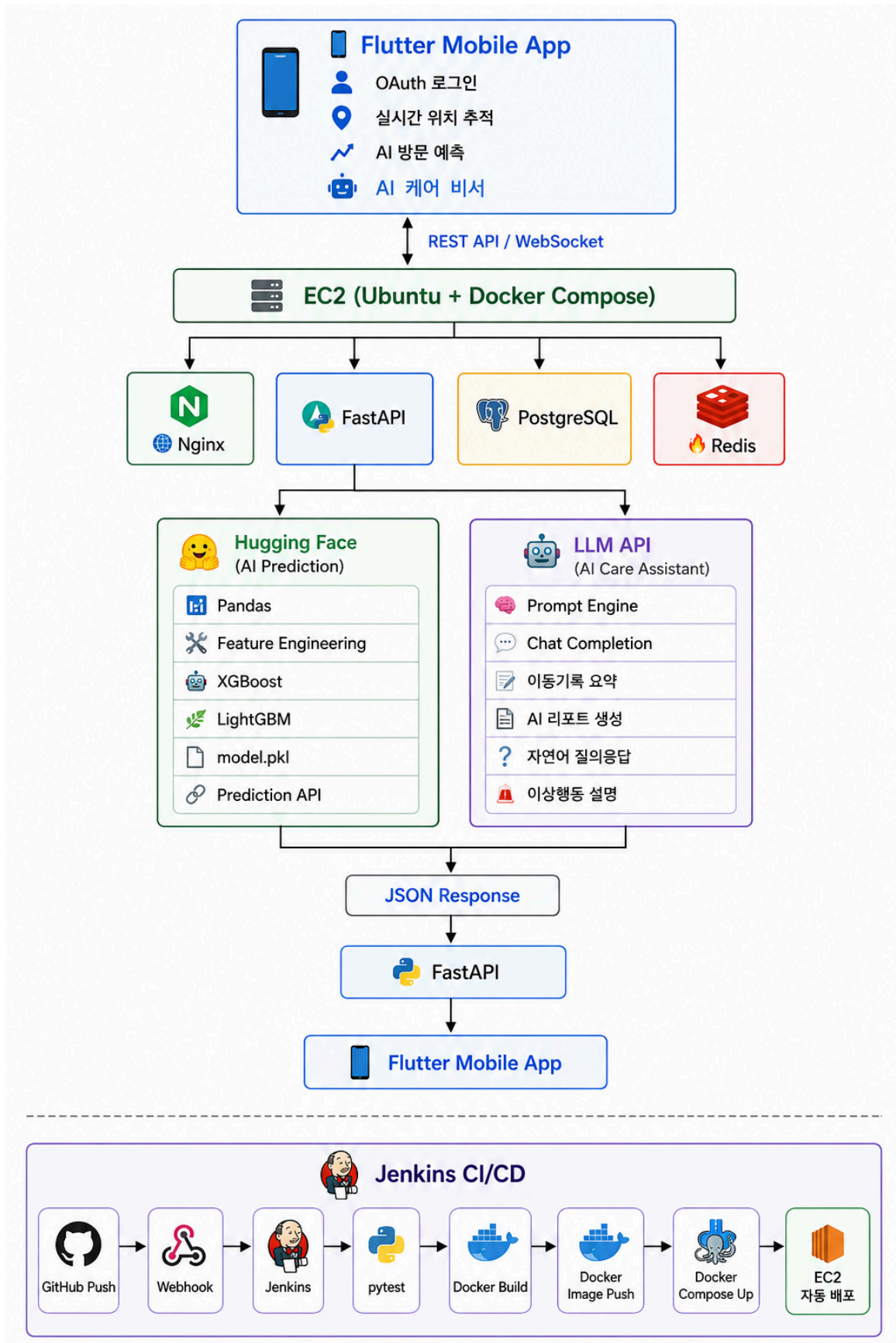
```

```
        created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
  
        CONSTRAINT fk_ch_user FOREIGN KEY (user_id) REFERENCES "User"(id) ON DELETE CASCADE  
    );
```

데이터베이스 SQL문_init.sql

실서버 배포 시, 비용 최소화 구성 (보류)

▼ 컴퓨터 사양 고려 아키텍처



▼ 역할 분리

▼ EC2 - 가상서버 1대 (무료티어) / 도메인(선택) - 비용 (o)

비즈니스 로직 :

로그인 / OAuth / JWT / 위치 저장 / GeoFence / WebSocket / FCM / Redis 캐시

▼ Hugging Face

AI 학습과 예측

- 허깅페이스가 **sleep 상태일 경우**를 대비시 Redis 캐시를 활용해 선 조회 후 없으면 허깅페이스 예측 실행

▼ Docker

nginx / fastAPI / postgres / redis / jenkins

• 정리

- **EC2(Docker Compose)** : FastAPI + PostgreSQL + Redis + Jenkins + Nginx
- **Hugging Face Space** : Pandas 전처리 + XGBoost/LightGBM 학습 및 예측
- **Redis** : AI 예측 결과 캐시
- **Jenkins** : GitHub Webhook 기반 자동 배포 (CD 까지 구현할 경우)

• 구조화의 장점

개발 단계 : 비용 절감을 위해 **Docker Volume** 에 이미지 저장하여 구현

운영 단계 : S3 + CloudFront로 쉽게 확장 가능한 구조

